
UNIT 12 MINING DATA STREAMS

Structure

12.0 Introduction

12.1 Objectives

12.2 Data Streams

12.2.1 Model for Data Stream Processing

12.3 Data Stream Management

12.3.1 Queries of Data Stream

12.3.2 Examples of Data Stream and Queries

12.3.3 Issues and Challenges of Data Stream

12.4 Summary

12.5 Answers

12.6 References/Further Readings

12.0 INTRODUCTION

In today's data-driven world, vast amounts of data are generated continuously from various sources such as sensors, online transactions, social media platforms, and network monitoring systems. This type of continuously flowing data is referred to as a **data stream**. Unlike traditional datasets that are static and stored in databases, data streams are dynamic, unbounded, and arrive at high speed, requiring immediate processing and analysis.

To understand and work with data streams effectively, it's important to explore a **model for data stream processing**, which outlines how incoming data can be captured, processed, and analyzed in real time. Such models typically focus on minimizing memory usage, ensuring quick computation, and providing approximate but useful results.

Managing these data streams is equally important and is addressed under the concept of **Data Stream Management**. In this context, learners should understand how to write **queries for data streams**, which differ from traditional SQL queries because of the continuous and time-sensitive nature of the data. Real-life **examples of data streams and related queries** help illustrate how theoretical concepts are applied in practical scenarios like stock market analysis or network traffic monitoring.

However, working with data streams also brings several **issues and challenges**. These include coping with limited memory, handling out-of-order data, processing at high speeds, and generating timely insights. Understanding these challenges is essential for designing efficient data stream processing systems.

By studying the concepts of data stream models, management techniques, and the practical challenges involved, learners can develop the skills necessary to work with real-time data in various modern applications.

This unit explains the characteristics and models of data stream processing. It also identifies the challenges of stream processing and introduces you to various techniques for processing of data streams. You may refer to further readings of this Unit for more details on data streams and data stream processing.

12.1 OBJECTIVES

After going through this Unit, you would be able to:

- Define the characteristics of data streams
- Discuss models of data stream processing
- Explain the uses of data streams
- Illustrate the example of data stream queries
- Address Issues and Challenges of Data Stream

12.2 DATA STREAMS

In many traditional applications, data is stored in a **database** or a **distributed file system**, allowing users to access the same data multiple times whenever needed. This works well when the data is relatively stable and can be stored for long-term use. However, there are certain real-world scenarios where this approach is not practical. For example, in **real-time applications** like data received from **weather satellites**, the volume and speed of incoming data are so high that storing all of it in a database becomes either **impractical or inefficient**.

In some cases, the data might not be stored at all, or if it is, the **database may be so large** that retrieving information quickly enough to make timely decisions becomes impossible. This is where traditional query methods fall short. To deal with such situations, we need **new techniques** that allow us to process and analyze data on the fly.

To address this, we use **approximation techniques** and **data compression methods** that help us extract useful insights from the data without needing to store or scan the entire dataset. These methods aim to answer the most important queries **quickly and efficiently**, often with a trade-off between accuracy and performance.

This brings us to the concept of **data stream mining**—a process of extracting meaningful knowledge from **continuous, high-speed, and volatile data streams** in **real time**. The term "volatile" here means that the data is constantly changing and evolving, and due to resource constraints (like memory or processing power), we cannot store it all permanently. Instead, we must process it as it arrives, making **real-time analysis** a necessity.

So, how can we analyze such fast and endless streams of data effectively? To explore this question, the next section introduces the **models for data stream processing**, which provide structured approaches for handling, analyzing, and querying data streams under real-world constraints.

12.2.1 Model for Data Stream Processing

Let's Understanding the Model for Data Stream Processing, when working with traditional datasets, we usually have the entire data available beforehand, stored in a database or a file system. We can query it multiple times, sort it, scan it repeatedly, or even join it with other data. But in data stream processing, things are different. The data arrives continuously, often at high speed, and we only get one chance to process it. This is because storing the entire stream is not practical due to limited memory, time, and processing power.

To deal with this challenge, researchers and developers have come up with specific models for processing data streams efficiently. Each model provides a strategy to analyse and summarize the incoming data using minimal resources, often with a trade-off in accuracy. Let's explore some of the commonly used models with easy-to-understand examples.

1. **Sliding Window Model** : In this model, we focus only on the **most recent data** in the stream. Instead of processing the entire stream, we maintain a “**window**” of data (either time-based or count-based), and continuously update our analysis as new data enters and old data exits the window.

Example:

Imagine monitoring **temperature readings** from a sensor every second. Using a sliding window of the **last 10 minutes**, we can calculate the **average temperature** in that window. As new readings arrive, old ones are discarded, and the average is updated in real time.

2. **Damped Window Model (Exponential Decay Model)**: In this model, **recent data is given more importance** than older data. Rather than maintaining a fixed-size window, we use a mathematical function (usually exponential decay) to reduce the weight of older data gradually.

Example:

In **stock market trend analysis**, recent price changes are more relevant than older ones. Using the damped window model, we can calculate a **weighted average price**, where the latest data points have the highest influence on the result.

3. **Sampling Model**: Here, instead of analysing the entire stream, we **randomly select a subset** (sample) of the data and use it to make estimations. This reduces memory and computation requirements significantly. **Example:** If we are monitoring **website traffic** from millions of users per day, we might randomly sample 1% of user activity to estimate **popular pages or average session time**, instead of analysing all traffic.
4. **Sketching Model**: This model uses compact data structures called **sketches** to summarize large data streams. Sketches provide **approximate answers** to queries like frequency count or cardinality (number of unique items), using very little memory. **Example:** A network router might use sketching to **count the number of unique IP**

addresses accessing it in real time, without storing every single IP. One popular sketching technique is the **Count-Min Sketch**.

5. **Aggregation Model:** In this model, the data stream is processed using **aggregate functions** like sum, count, average, min, or max. This model is useful for summarizing the data continuously. **Example:** A smart energy meter may use aggregation to calculate the **total electricity consumption** every hour by continuously adding up the readings from the data stream.

These models help us handle the unique challenges of data stream processing. Depending on the application—whether it’s real-time weather monitoring, network traffic analysis, or sensor-based automation—we can choose the most suitable model to process data efficiently. As a learner, understanding these models equips you to design systems that can make fast, resource-efficient decisions from rapidly changing data.

You may use any model for data stream processing, but how are data streams managed? Next section discusses the data stream management system.

12.3 DATA STREAM MANAGEMENT

Managing data streams is a critical task in today’s data-centric world, especially when dealing with real-time applications like traffic monitoring, stock trading, or weather forecasting. Unlike traditional data stored in databases, data streams are continuous, fast, and often infinite. This means we usually don’t have the luxury to store all the incoming data or process it multiple times. So, how do we effectively manage such data? The answer lies in using smart models that help us process, analyze, and summarize data streams efficiently, often with limited memory and computing resources.

One useful approach is the **sliding window model**, where only the most recent portion of the data stream is kept in memory. For example, if we are monitoring traffic at a busy intersection, we might only care about vehicle counts in the last 5 minutes, rather than the entire day. As new data arrives, old data is removed from the window, allowing us to continuously analyze the latest situation without overloading the system.

Another important model is the **damped window model**, which helps when we still want to consider older data, but give more importance to recent values. This is especially helpful in applications like fraud detection, where past user behavior provides useful context, but recent actions are more indicative of suspicious activity. In this model, the influence of older data gradually fades, allowing the system to focus on current trends while still acknowledging historical patterns.

In cases where the data stream is too fast or too large to process entirely, we can use the **sampling model**. This approach involves selecting a small, random subset of data to analyze, which still gives us a fairly accurate understanding of the overall trend. For example, a video streaming service may sample a small percentage of user viewing data to identify popular content, rather than analyzing every single view.

For even more compact summaries, the **sketching model** comes in handy. Sketching uses clever data structures to keep track of key statistics—like the number of unique users or the most frequent search terms—without storing the entire stream. This is very useful in network monitoring or cybersecurity, where we might need to detect abnormal behavior in real time using only minimal memory.

Finally, the **aggregation model** is ideal for continuously calculating values like totals, averages, or maximums. For instance, an electricity provider may use this model to calculate the total energy consumed by each household every hour. The aggregation is done on-the-fly, which helps make timely decisions such as load balancing or dynamic billing.

So, we learned that, managing data streams effectively means choosing the right processing model based on the type of analysis required and the limitations of the system. Sliding and damped windows help focus on the most relevant data, sampling reduces processing load, sketching offers lightweight summaries, and aggregation delivers real-time metrics. By understanding and applying these models thoughtfully, we can build systems that turn continuous data into meaningful insights—even in the face of speed, scale, and complexity. In general, such systems are known as Data Stream Management Systems (DSMS).

In today's world, where data is generated continuously from sources like sensors, social media, mobile devices, and financial transactions, managing this constant flow of information becomes a challenge. Traditional database systems are not designed to handle such high-speed, real-time data. This is where a **Data Stream Management System (DSMS)** comes into play.

The Data Stream Management System (DSMS) extracts knowledge from multiple data streams by eliminating undesirable elements, as shown in Figure 1. DSMS is important where the input data rate is controlled externally. For example, Google queries.

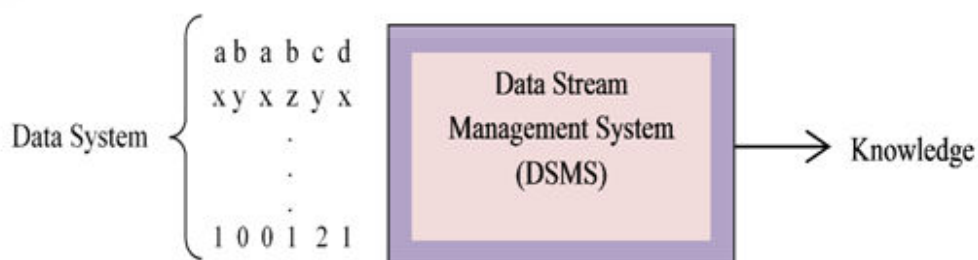


Figure 12.1: A simple outline of Data Stream Management System

The sources of data streams include Internet traffic, online transactions, satellite data, sensors data, live event data, real-time surveillance systems, etc.

So, we can say that a **Data Stream Management System** is a specialized software system designed to **process, manage, and analyze data streams in real time**. Unlike traditional database management systems (DBMS), which deal with static, stored data, a DSMS handles **dynamic and transient data**—data that arrives continuously and may not be stored permanently.

Thus we can think of a DSMS as a **real-time engine** that captures live data, runs queries on it as it flows in, and produces immediate results. It allows us to perform actions like filtering, aggregating, joining, or detecting patterns **on the fly** without storing the entire dataset.

For example, imagine a weather monitoring station that receives temperature, humidity, and wind speed data every few seconds from hundreds of sensors. A traditional system would store all this data first and then analyze it later. But a DSMS processes this data **as it arrives**, allowing meteorologists to instantly detect unusual weather conditions, such as a sudden drop in temperature or wind speed spikes.

A DSMS typically includes several key components:

- **Data Stream Input Interface** – It connects with data sources like sensors, logs, or applications to receive the stream.
- **Query Processor** – It continuously runs user-defined queries (like SQL) over the data stream.
- **Memory Manager** – Since storing all data isn't possible, it uses memory efficiently to store only what's needed, often using techniques like sliding windows or summaries.
- **Scheduler and Optimizer** – These components ensure that queries run quickly and efficiently, even under heavy data loads.
- **Output Interface** – Sends the results to dashboards, alerting systems, or other applications for further action.

Some popular DSMS tools include **Apache Flink**, **Apache Storm**, **Spark Streaming**, and **Google Dataflow**, which are widely used in industries ranging from finance to healthcare to smart cities.

Figure 2 shows the detailed view of a data stream management system. The components of this system are described as follows:

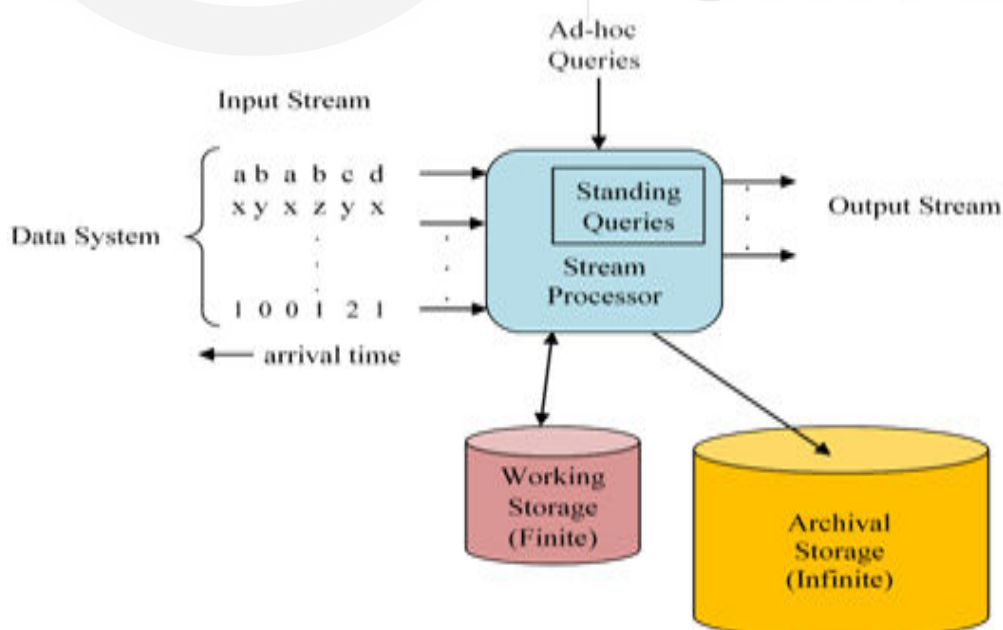


Figure 12.2: A detailed view of Data Stream Management System

- **Processor:** The processor is a software that executes the queries on the data stream. There can be multiple processors working together. The processor may store some standing queries and also allows ad-hoc queries (refer to section 10.3.1) to be issued by the users.
- **Streams Entering:** There can be several streams entering in the system. Conventionally, we will assume that the elements at the right end of the stream have arrived more recently. The time goes backward to the left i.e. the further left the earlier the element entered the system.
- **Output:** The system makes output in response to the standing queries and the ad-hoc queries (refer to section 10.3.1).
- **Archival Storage:** There is a massive archival storage and we cannot assume the archival storage is architected like a database system. Further, we can use appropriate indices or other tools to efficiently answer the queries from that data. We only know that if we had to reconstruct the history of the streams it could take a long time.

Limited Working Storage: It might be a main memory, or a flash storage, or even a magnetic disk. But we assume that it holds important parts of the input streams in a way that supports fast execution of query.

In short, learning about DSMS is essential if you want to build systems that work with real-time data. It enables **faster decision-making**, supports **live analytics**, and helps us keep up with the **fast-paced digital environment** we live in. As a learner, try exploring how DSMS tools work in real-world applications like fraud detection, traffic monitoring, and online recommendation systems to understand their power and importance.

There is a **fundamental difference between Data Base Management System (DBMS) and Data Stream Management System (DSMS)** is who controls how data enters into the system.

(a) DBMS:

In a DBMS, the staff associated with the management of the database usually insert data into the system using a bulk loader or even explicit SQL INSERT commands. The staff can decide how much data to be loaded into the system. The staff can also decide when and how fast to load the data into the system. *For example*, the records of students stored in schools or colleges or universities.

(b) DSMS:

In a DSMS, the management cannot control the rate of input. *For example*, the search queries that arrive at Google search engine are generated by random people around the globe, who search for information at their respective pace. Google staff literally has no control over the rate of arrival of queries. They have to design and architect their system in such a way that it can easily deal with the varying data rate.

Next we need to understand, how to make queries of data streams, which is discussed next.

12.3.1 Queries Of Data Stream

When working with traditional databases, we write queries that run on stored data and return results after the computation is complete. But in the world of data streams, where data is constantly flowing in real time, we need a different kind of approach to querying. Here, the data is not static—it's live, fast, and potentially endless. This is where understanding how queries work in data streams becomes really important.

In data stream systems, we mainly deal with two types of queries: *adhoc* queries and *standing* (or *continuous*) queries.

Adhoc Queries: Adhoc queries are similar to traditional database queries. They are executed once on a snapshot or current state of the data (usually within a defined window). These queries are useful when we want to check something quickly or perform a one-time analysis. Example: Imagine we want to know how many users visited a website in the last 10 minutes. We can run an *adhoc* query over that time window to get the result once.

Standing (Continuous) Queries: Standing queries, also called continuous queries, are much more powerful and common in data stream systems. These queries are always active—they run continuously on the incoming stream and keep updating the result in real time as new data arrives. Example: If we want to monitor the average CPU usage of a server in real time and raise an alert if it goes above 80%, we'd use a standing query that constantly watches the stream of CPU data.

Common Types of Data Stream Queries: Whether *adhoc* or continuous, data stream queries are used to perform different operations such as:

- **Filtering:** Selecting data items that match a condition. *E.g., "Select all tweets containing the word 'earthquake'."*
- **Aggregation:** Calculating summaries like count, sum, average, max, or min, usually within a time window. *E.g., "Calculate the number of transactions every 30 seconds."*
- **Joins:** Combining two or more data streams based on a common key. *E.g., "Join GPS and fuel sensor streams to analyze fuel usage per route."*
- **Pattern Detection:** Identifying specific sequences or events. *E.g., "Detect three consecutive login failures from the same account."*

Important: One of the biggest differences between stream and traditional queries is that stream queries don't wait for the data to stop coming—they work on it as it flows. That's why standing queries are so important. They allow us to set up logic that keeps running in the background, continuously updating results or triggering actions. Meanwhile, *adhoc* queries give us the flexibility to ask one-time questions over a specific period or sample from the stream.

12.3.2 Examples of Data Stream

To understand what data streams really are, it helps to look at real-world situations where data isn't stored first, but instead flows in continuously. A data stream is a live, ongoing flow of information that must be processed in real time. This kind of data appears in many everyday applications, and recognizing these can help you better appreciate how streaming systems work.

1. Sensor Data Streams

One of the most common sources of data streams is sensors. These are used in everything from weather stations to smart homes and industrial machines. For example, weather sensors stream temperature, humidity, and wind speed data every few seconds. Similarly, smart home devices like thermostats or motion detectors constantly send activity updates. In industries, IoT-enabled machines send data about vibration, pressure, or speed to help monitor equipment health and prevent breakdowns.

2. Social Media Feeds

Social media platforms are full of streaming data. Every time someone posts a tweet, uploads a video, or reacts to a story, that action becomes part of a data stream. For example, Twitter generates a live stream of tweets that can be analyzed in real time to detect trends, sentiments, or viral content. Live comment sections during events or trending hashtags are other examples of this constantly flowing data.

3. Stock Market Data

Financial markets depend heavily on real-time data streams. Stock prices update thousands of times per second, and trading systems process buy/sell orders instantly. This stream of data helps investors make quick decisions and allows financial firms to detect patterns or anomalies, such as unusual trading activity that might indicate market manipulation or fraud.

4. Clickstream Data

When you browse a website or use an app, every action you take—like clicking a link, scrolling a page, or hovering over a product—is recorded. This flow of user behavior is called clickstream data. Companies like Amazon or Netflix analyze this data in real time to personalize your experience by recommending products or shows based on what you're doing right now, not just past behavior.

5. Telecommunication Streams

Telecom companies handle a huge amount of streaming data. Each phone call, message, or internet request is recorded and processed as it happens. For instance, call detail records (CDRs) are streamed to billing systems and fraud detection tools. Similarly, network performance data streams help ensure smooth service delivery and detect problems like dropped calls or slow internet speeds.

6. Video and Audio Streams

Any time you watch a live video or listen to music online, you're using a data stream. Platforms like YouTube Live, Zoom, or Spotify stream data in small chunks to your device continuously. These streams are processed in real time for quality control, buffering, and personalization, like adjusting the video resolution based on your network speed.

7. Traffic Monitoring Systems

Modern cities rely on traffic monitoring systems that collect real-time data from GPS devices, cameras, and road sensors. For example, GPS data from thousands of vehicles is streamed to control centers to detect traffic jams, plan signal timings, or suggest alternate routes. This helps improve traffic flow and reduce congestion on the roads.

8. E-commerce Transactions

Large online retailers generate massive data streams from user actions on their websites. Each product view, search, purchase, or payment is captured instantly. These streams are used to update inventory in real time, provide personalized recommendations, or flag suspicious transactions for fraud analysis.

To better understand data streams, look around you. Your fitness app counting steps, Google Maps updating traffic, or even your smart electric meter—all are working with data streams. As you begin to recognize these examples in everyday life, you'll start to see how important real-time data is in making systems smart, responsive, and efficient.

12.3.3 Issues and Challenges of Data Stream

As we explore the world of data stream processing, you'll quickly realize that it's very different from working with traditional static data. In streaming, data flows continuously, and we must process it in real time. While this offers exciting possibilities—like instant decision-making and live monitoring—it also introduces a number of technical and conceptual challenges. Let's break these down into key categories so that you can better understand the landscape.

1. **Continuous and Unbounded Data:** The most obvious challenge is that data streams are infinite—they never stop. Unlike static datasets, which have a definite beginning and end, stream data keeps coming. This makes it difficult to store all the data or reprocess it later. As a learner, ask yourself: how would you handle a firehose of data that never ends? The solution lies in using sliding or tumbling windows, sampling, or approximation techniques.
2. **Limited Memory and Storage:** Because stream processing needs to happen in real time and memory is limited, you can't store all incoming data. Systems must work with bounded resources—only keeping what's essential and discarding the rest. This means designing algorithms that can process and analyze data on the fly, often using summaries, counters, or sketches that give quick, approximate answers.

3. **Real-Time Query Processing:** Stream queries must run continuously and update results dynamically as new data arrives. Unlike traditional queries that run once and return a result, here the queries live alongside the data. This adds complexity: you need efficient ways to update the answers without reprocessing the entire stream. Try to understand how standing (continuous) queries and windowing functions help solve this challenge.
4. **Data Heterogeneity and Integration:** In many cases, data streams come from multiple, diverse sources. For example, a smart city application might receive streams from sensors, traffic cameras, and weather stations—all in different formats. Managing and integrating this heterogeneous data in real time requires strong data transformation and preprocessing techniques. As a learner, think about how you'd normalize or clean such data before it flows into your analysis pipeline.
5. **Noisy, Incomplete, or Inaccurate Data:** Stream data often arrives with errors or missing values, and there's no time for complex cleaning processes. You must learn to work with imperfect data and still extract meaningful insights. Systems often use filtering, approximation, or confidence thresholds to deal with such challenges. Recognizing this helps you build resilient systems that don't break under messy inputs.
6. **Out-of-Order Data and Latency:** One of the trickier issues in data streaming is when data doesn't arrive in the same order it was generated. For example, a sensor update might be delayed due to a network glitch. This causes out-of-order events, which can affect accuracy. Handling this requires event time processing, watermarks, and buffering strategies. As you go deeper into stream processing tools, look at how they manage time and order.
7. **Security and Privacy:** Many data streams include sensitive or personal information—think of financial transactions, GPS locations, or health data. Ensuring that this data remains secure, and is handled in compliance with privacy laws (like GDPR), is critical. As a learner, consider how encryption, access control, and anonymization might be applied in real-time systems.

Understanding these challenges is the first step to designing robust and scalable data stream systems. Each challenge teaches you something new about real-time computing. Try exploring open-source platforms like **Apache Kafka**, **Apache Flink**, or **Apache Spark Streaming** to see how these issues are solved in practice. With hands-on practice, you'll begin to appreciate the balance between **speed**, **accuracy**, and **efficiency** in stream processing.

Check Your Progress 1:

1. Define the data stream processing. Which model of data stream processing is useful in finding stock market trends? Justify.

.....
.....

2. Differentiate between DBMS and DSMS. Why is all the data of data streams not stored?

.....

.....

.....

3. How are Standing queries different to ad-hoc queries?

.....

.....

.....

4. What is the use of mining click streams?

.....

.....

.....

12.4 SUMMARY

This unit introduces the concept of mining data streams. Data streams can be processed using different models. Three such models (landmark, sliding windows, and damped model) for data stream processing are introduced in this unit. Further, the data stream management system (DSMS) is explained. In addition, different types of queries of data streams namely, ad-hoc and standing queries are discussed followed by examples of these queries. The issues and challenges of Data streams and data sampling in data streams with examples of representation sample and sampling Key-Value pairs are discussed in this unit.

12.5 ANSWERS

Check Your Progress 1:

1. The data stream is a process to extract knowledge in real time from a large amount of volatile data, which comes in an infinite stream of data. The data is volatile because it is continuously changing and evolving over time. The system does not store the data in the database due to limited resources. The landmark model of data stream processing is useful in finding stock market trends because it finds the frequently used items in the entire data stream from a specific time till present and all time-points are treated equally after the starting time.
2. In a DBMS, the staff associated with the management of the database usually insert data into the system. In a DSMS, the management cannot control the rate of input. All data from data streams is not stored due to a limited amount of resources.
3. In standing queries, the users expect the system to report the answer available at all times, perhaps outputting a new value each time the answer changes. In ad-hoc queries, the users make a query once and

expect an answer to the query based on the current state of the system, which is available only at current time.

4. The use of mining click streams is to know which URLs are getting clicked-on, a lot more, in the past one hour than normal.

12.6 REFERENCES/FURTHER READINGS

- [1] Mansalis, Stratos, et al. "An evaluation of data stream clustering algorithms." *Statistical Analysis and Data Mining: The ASA Data Science Journal* 11.4 (2018): 167-187
- [2] Albert C. "Introduction to Stream Mining."
<https://towardsdatascience.com/introduction-to-stream-mining-8b79dd64e460>
- [3] "Mining Data Streams."
<http://infolab.stanford.edu/~ullman/mmds/ch4.pdf>

